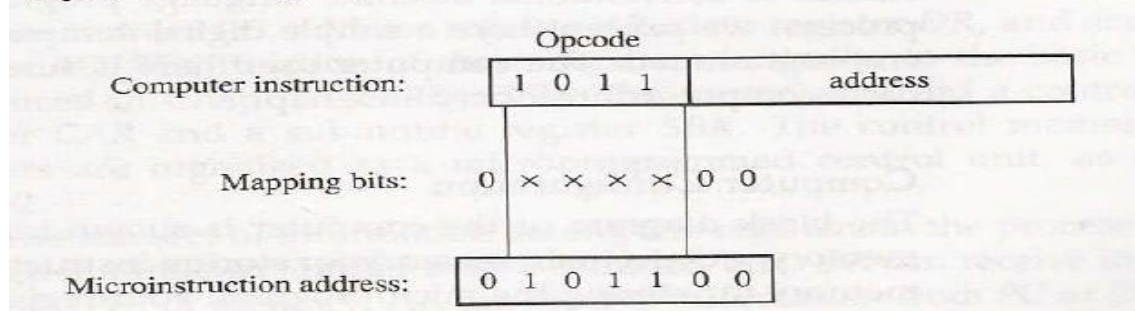


- The branch logic tests the condition, if met then branches, otherwise, increments the CAR.
- If there are 8 status bit conditions, then 3 bits in the microinstruction are used to specify the condition and provide the selection variables for the multiplexer.
- For unconditional branching, fix the value of one status bit to be one load the branch address from control memory into the CAR.

Mapping of Instruction:

- A special type of branch exists when a microinstruction specifies a branch to the first word in control memory where a microprogram routine is located.
- The status bits for this type of branch are the bits in the opcode.
- Assume an opcode of four bits and a control memory of 128 locations. The mapping process converts the 4-bit opcode to a 7-bit address for control memory shown in below figure.

Figure 7-3 Mapping from instruction code to microinstruction address.



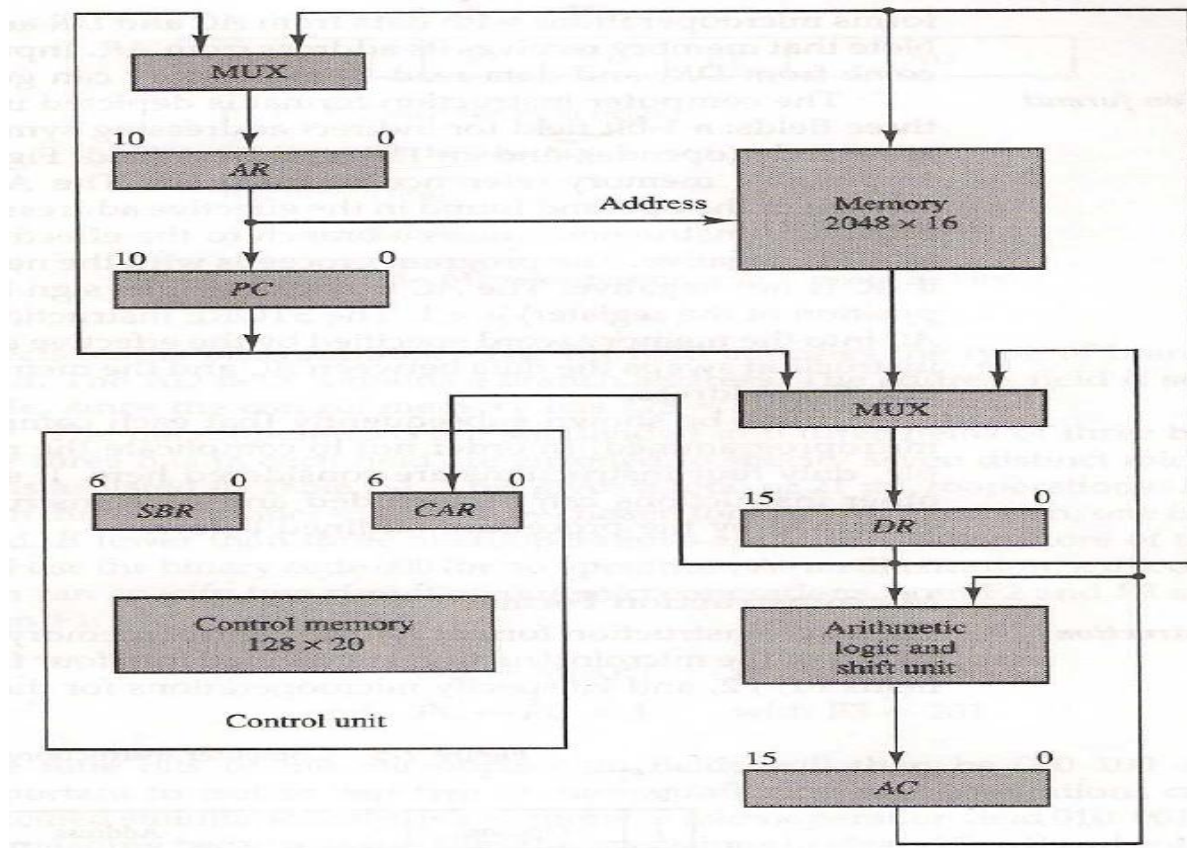
- Mapping consists of placing a 0 in the most significant bit of the address, transferring the four operation code bits, and clearing the two least significant bits of the control address register.
- This provides for each computer instruction a microprogram routine with a capacity of four microinstructions.

Subroutines:

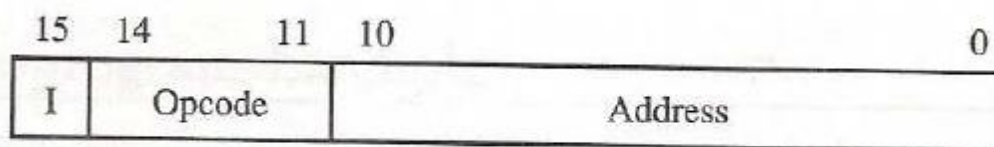
- Subroutines are programs that are used by other routines to accomplish a particular task and can be called from any point within the main body of the microprogram.
- Frequently many microprograms contain identical section of code.
- Microinstructions can be saved by employing subroutines that use common sections of microcode.
- Microprograms that use subroutines must have a provision for storing the return address during a subroutine call and restoring the address during a subroutine return.
- A subroutine register is used as the source and destination for the addresses

3. Microprogram Example:

- The process of code generation for the control memory is called *microprogramming*.
- The block diagram of the computer configuration is shown in below figure.
- Two memory units:
 - Main memory – stores instructions and data
 - Control memory – stores microprogram
- Four processor registers
 - Program counter – PC
 - Address register – AR
 - Data register – DR
 - Accumulator register - AC
- Two control unit registers
 - Control address register – CAR
 - Subroutine register – SBR
- Transfer of information among registers in the processor is through MUXs rather than a bus.



- The computer instruction format is shown in below figure.



(a) Instruction format

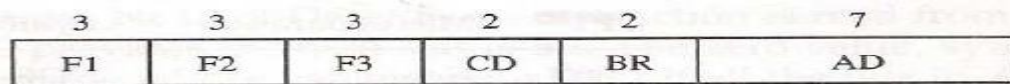
- Three fields for an instruction:
 - 1-bit field for indirect addressing
 - 4-bit opcode
 - 11-bit address field
- The example will only consider the following 4 of the possible 16 memory instructions

Symbol	Opcode	Description
ADD	0000	$AC \leftarrow AC + M[EA]$
BRANCH	0001	If $(AC < 0)$ then $(PC \leftarrow EA)$
STORE	0010	$M[EA] \leftarrow AC$
EXCHANGE	0011	$AC \leftarrow M[EA], M[EA] \leftarrow AC$

EA is the effective address

(b) Four computer instructions

- The microinstruction format for the control memory is shown in below figure.



F1, F2, F3: Microoperation fields

CD: Condition for branching

BR: Branch field

AD: Address field

Figure 7-6 Microinstruction code format (20 bits).

- The microinstruction format is composed of 20 bits with four parts to it
 - Three fields F1, F2, and F3 specify microoperations for the computer [3 bits each]
 - The CD field selects status bit conditions [2 bits]
 - The BR field specifies the type of branch to be used [2 bits]
 - The AD field contains a branch address [7 bits]
- Each of the three microoperation fields can specify one of seven possibilities.
- No more than three microoperations can be chosen for a microinstruction.
- If fewer than three are needed, the code 000 = NOP.
- The three bits in each field are encoded to specify seven distinct microoperations listed in below table.

F1	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC + DR$	ADD
010	$AC \leftarrow 0$	CLRAC
011	$AC \leftarrow AC + 1$	INCAC
100	$AC \leftarrow DR$	DRTAC
101	$AR \leftarrow DR(0-10)$	DRTAR
110	$AR \leftarrow PC$	PCTAR
111	$M[AR] \leftarrow DR$	WRITE

F2	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC - DR$	SUB
010	$AC \leftarrow AC \vee DR$	OR
011	$AC \leftarrow AC \wedge DR$	AND
100	$DR \leftarrow M[AR]$	READ
101	$DR \leftarrow AC$	ACTDR
110	$DR \leftarrow DR + 1$	INCDR
111	$DR(0-10) \leftarrow PC$	PCTDR

F3	Microoperation	Symbol
000	None	NOP
001	$AC \leftarrow AC \oplus DR$	XOR
010	$AC \leftarrow \overline{AC}$	COM
011	$AC \leftarrow \text{shl } AC$	SHL
100	$AC \leftarrow \text{shr } AC$	SHR
101	$PC \leftarrow PC + 1$	INCP
110	$PC \leftarrow AR$	ARTPC
111	Reserved	

- Five letters to specify a transfer-type microoperation
 - First two designate the source register
 - Third is a 'T'
 - Last two designate the destination register $AC \leftarrow DR$ F1 = 100 = DRTAC
- The condition field (CD) is two bits to specify four status bit conditions shown below

CD	Condition	Symbol	Comments
00	Always = 1	U	Unconditional branch
01	$DR(15)$	I	Indirect address bit
10	$AC(15)$	S	Sign bit of AC
11	$AC = 0$	Z	Zero value in AC

- The branch field (BR) consists of two bits and is used with the address field to choose the address of the next microinstruction.

- The content of CAR is incremented and applied to one of the multiplexer inputs and to the subroutine register *SBR*.
- The other three inputs to multiplexer come from
 - The address field of the present microinstruction
 - From the out of *SBR*
 - From an external source that maps the instruction
- The *CD* (condition) field of the microinstruction selects one of the status bits in the second multiplexer.
- If the bit selected is equal to 1, the *T* variable is equal to 1; otherwise, it is equal to 0.
- The *T* value together with two bits from the *BR* (branch) field goes to an input logic circuit.
- The input logic in a particular sequencer will determine the type of operations that are available in the unit.

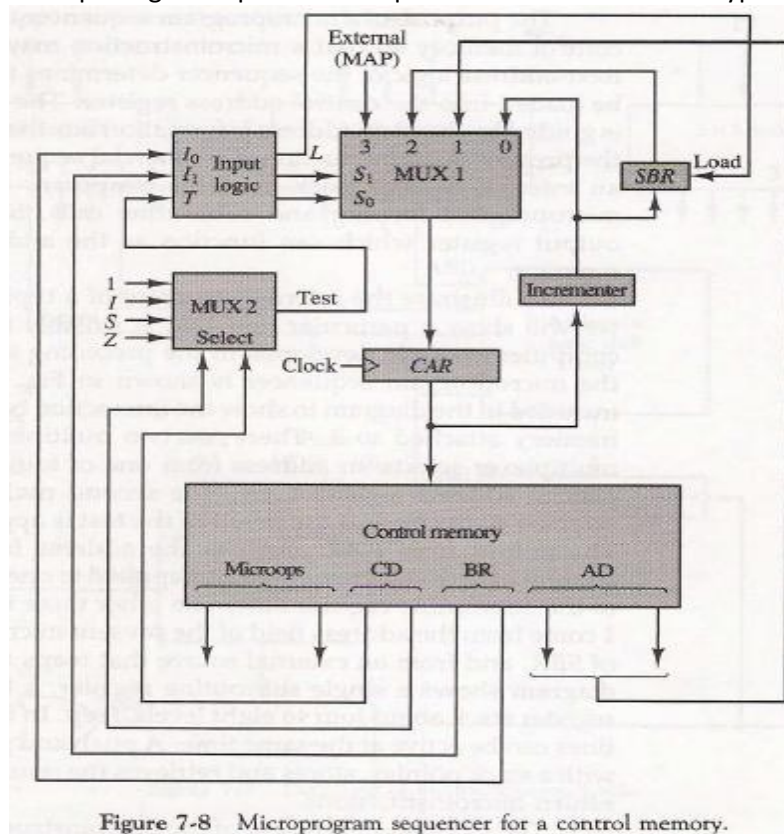


Figure 7-8 Microprogram sequencer for a control memory.

- The input logic circuit in above figure has three inputs I_0 , I_1 , and T , and three outputs, S_0 , S_1 , and L .
- Variables S_0 and S_1 select one of the source addresses for *CAR*. Variable L enables the load input in *SBR*.
- The binary values of the selection variables determine the path in the multiplexer.
- For example, with $S_1, S_0 = 10$, multiplexer input number 2 is selected and establishes transfer path from *SBR* to *CAR*.
- The truth table for the input logic circuit is shown in Table below.

BR Field		Input I_1 I_0 T			MUX 1 S_1 S_0		Load <i>SBR</i> L
0	0	0	0	0	0	0	0
0	0	0	0	1	0	1	0
0	1	0	1	0	0	0	0
0	1	0	1	1	0	1	1
1	0	1	0	×	1	0	0
1	1	1	1	×	1	1	0

- Inputs I_1 and I_0 are identical to the bit values in the *BR* field.
- The bit values for S_1 and S_0 are determined from the stated function and the path in the multiplexer that establishes the required transfer.
- The subroutine register is loaded with the incremented value of *CAR* during a call microinstruction (*BR* = 01) provided that the status bit condition is satisfied ($T = 1$).
- The truth table can be used to obtain the simplified Boolean functions for the input logic circuit:

$$\begin{aligned}
 S_1 &= I_1 \\
 S_0 &= I_1 I_0 + I_1 T \\
 L &= I_1 T I_0
 \end{aligned}$$

subtraction algorithm).

Addition and Subtraction of Signed-Magnitude Numbers

Computer Arithmetic

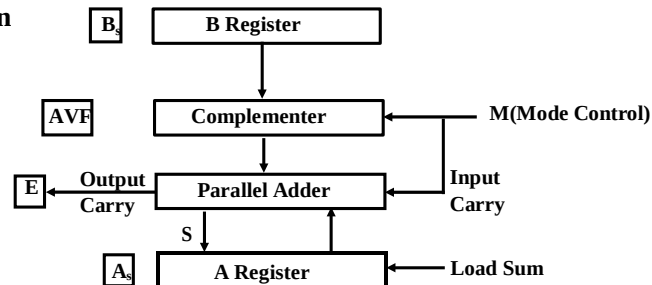
Addition and Subtraction

SIGNED MAGNITUDE ADDITION AND SUBTRACTION

Addition: $A + B$; A: Augend; B: Addend
Subtraction: $A - B$; A: Minuend; B: Subtrahend

Operation	Add Magnitude	Subtract Magnitude		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$			
$(+A) + (-B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (+B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A + B)$			
$(+A) - (+B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$			
$(-A) - (+B)$	$-(A + B)$			
$(-A) - (-B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$

Hardware Implementation

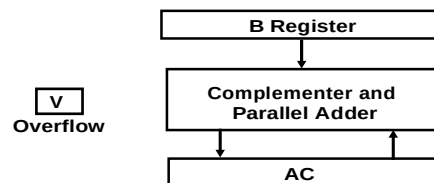


Computer Arithmetic

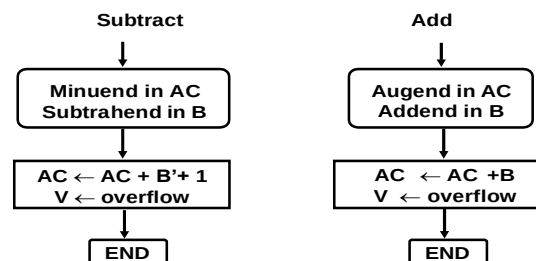
Addition and Subtraction

SIGNED 2'S COMPLEMENT ADDITION AND SUBTRACTION

Hardware



Algorithm



Algorithm:

- The flowchart is shown in Figure 7.1. The two signs A, and B, are compared by an exclusive-OR gate.

If the output of the gate is 0 the signs are identical; If it is 1, the signs are different.

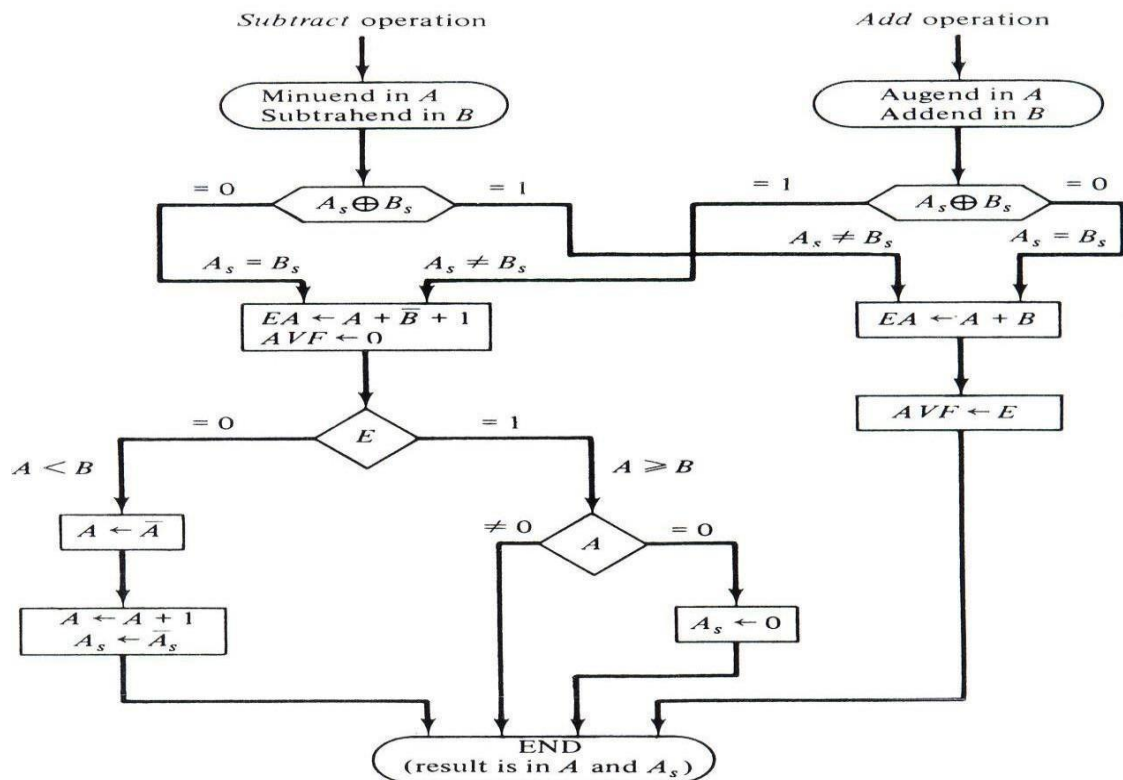
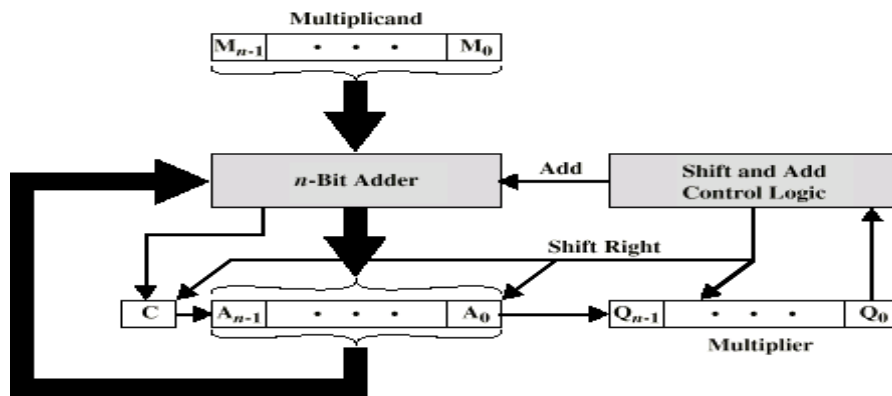


Figure 10-2 Flowchart for add and subtract operations.

Multiplication Algorithm:

In the beginning, the multiplicand is in B and the multiplier in Q. Their corresponding signs are in Bs and Qs respectively. We compare the signs of both A and Q and set to corresponding sign of the product since a double-length product will be stored in registers A and Q. Registers A and E are cleared and the sequence counter SC is set to the number of bits of the multiplier. Since an operand must be stored with its sign, one bit of the word will be occupied by the sign and the magnitude will consist of n-1 bits.

Now, the low order bit of the multiplier in Qn is tested. If it is 1, the multiplicand (B) is added to present partial product (A), 0 otherwise. Register EAQ is then shifted once to the right to form the new partial product. The sequence counter is decremented by 1 and its new value checked. If it is not equal to zero, the process is repeated and a new partial product is formed. When SC = 0 we stops the process.



C	A	Q	M	
0	0000	1101	1011	Initial Values
0	1011	1101	1011	Add
0	0101	1110	1011	Shift } First Cycle
0	0010	1111	1011	Shift } Second Cycle
0	1101	1111	1011	Add
0	0110	1111	1011	Shift } Third Cycle
1	0001	1111	1011	Add
0	1000	1111	1011	Shift } Fourth Cycle

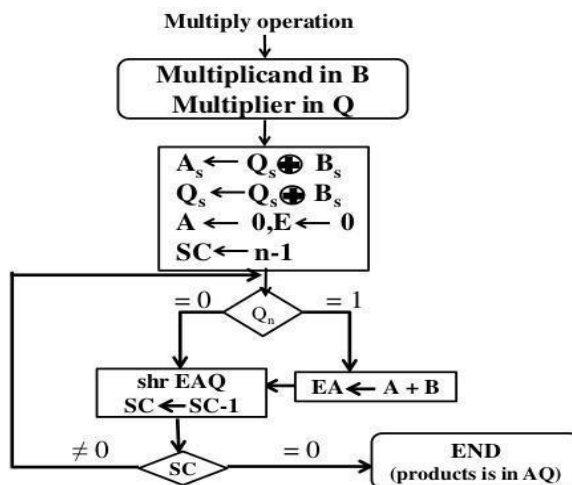


Figure: Flowchart for multiply operation.

Booth's algorithm :

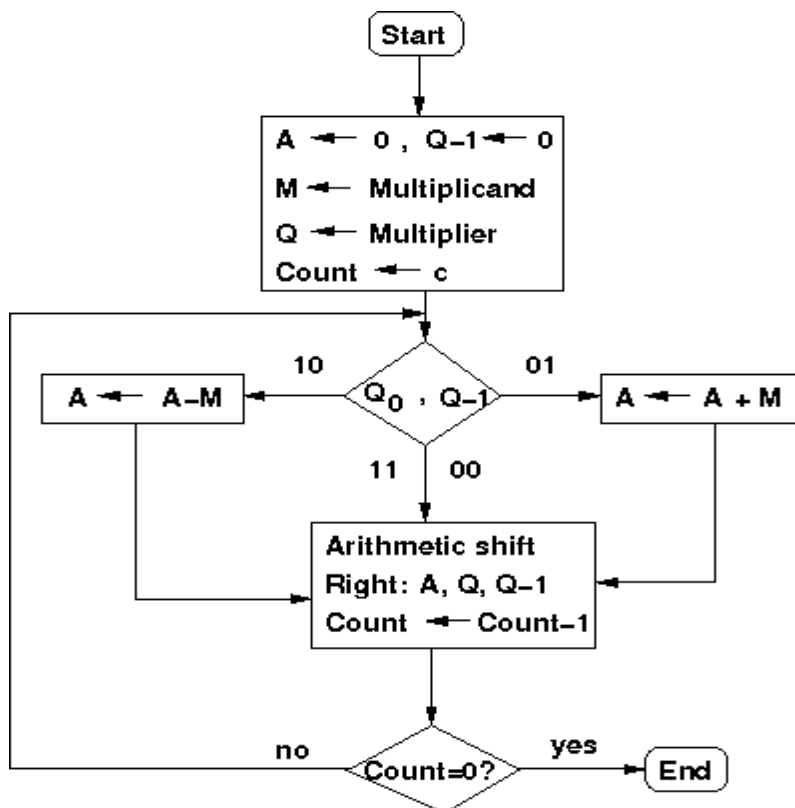
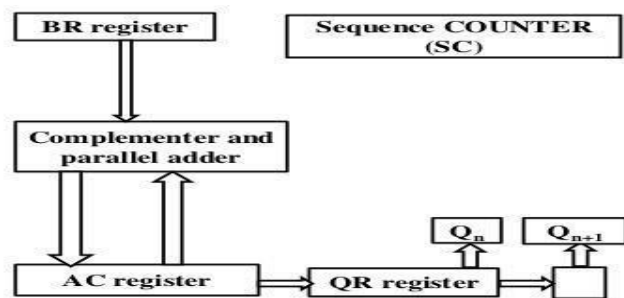
- Booth algorithm gives a procedure for multiplying binary integers in signed-2's complement representation.
- It operates on the fact that strings of 0's in the multiplier require no addition but just

shifting, and a string of 1's in the multiplier from bit weight 2^k to weight 2^m can be treated as $2^{k+1} - 2^m$.

- For example, the binary number 001110 (+14) has a string 1's from 2^3 to 2^1 ($k=3, m=1$). The number can be represented as $2^{k+1} - 2^m = 2^4 - 2^1 = 16 - 2 = 14$. Therefore, the multiplication $M \times 14$, where M is the multiplicand and 14 the multiplier, can be done as $M \times 2^4 - M \times 2^1$.
- Thus the product can be obtained by shifting the binary multiplicand M four times to the left and subtracting M shifted left once.

Hardware for Booth Algorithm

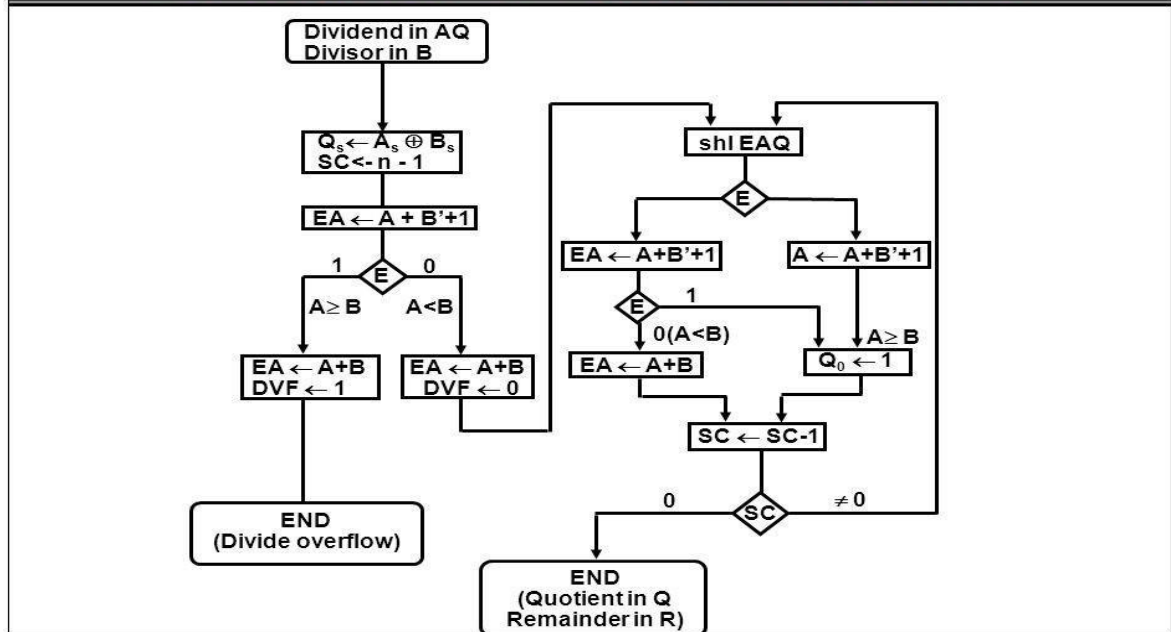
- Sign bits are not separated from the rest of the registers
- rename registers A, B, and Q as AC, BR and QR respectively
- Q_n designates the least significant bit of the multiplier in register QR
- Flip-flop Q_{n+1} is appended to QR to facilitate a double bit inspection of the multiplier



- As in all multiplication schemes, booth algorithm requires examination of the multiplier bits and shifting of partial product.

Algorithm:

FLOWCHART OF DIVIDE OPERATION



Example of Binary Division with Digital Hardware

Divisor B = 10001

	E	A	Q	SC
Dividend:		01110	00000	5
shl EAQ	0	11100	00000	
add $\bar{B} + 1$		01111		
E = 1	1	01011		
Set $Q_s = 1$	1	01011	00001	4
shl EAQ	0	10110	00010	
Add $\bar{B} + 1$		01111		
E = 1	1	00101		
Set $Q_s = 1$	1	00101	00011	3
shl EAQ	0	01010	00110	
Add $\bar{B} + 1$		01111		
E = 0; leave $Q_s = 0$	0	11001	00110	
Add B		10001		2
Restore remainder	1	01010		
shl EAQ	0	10100	01100	
Add $\bar{B} + 1$		01111		
E = 1	1	00011		
Set $Q_s = 1$	1	00011	01101	1
Shl EAQ	0	00110	11010	
Add $\bar{B} + 1$		01111		
E = 0; leave $Q_s = 0$	0	10101	11010	
Add B		10001		
Restore remainder	1	00110	11010	0
Neglect E				
Remainder in A:		00110		
Quotient in Q:			11010	

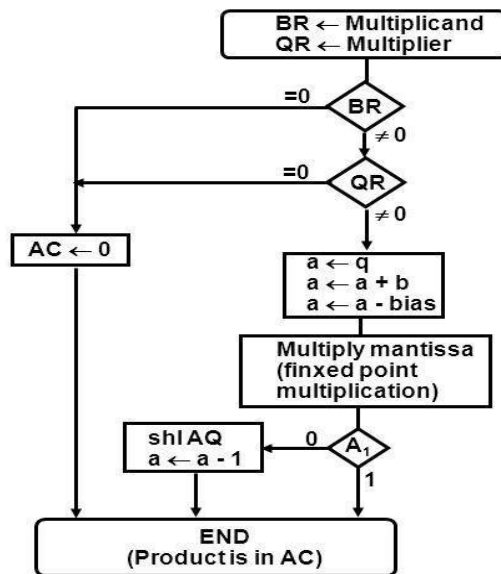
Multiplication:

Computer Arithmetic

16

Floating Point Arithmetic

FLOATING POINT MULTIPLICATION



Computer Organization

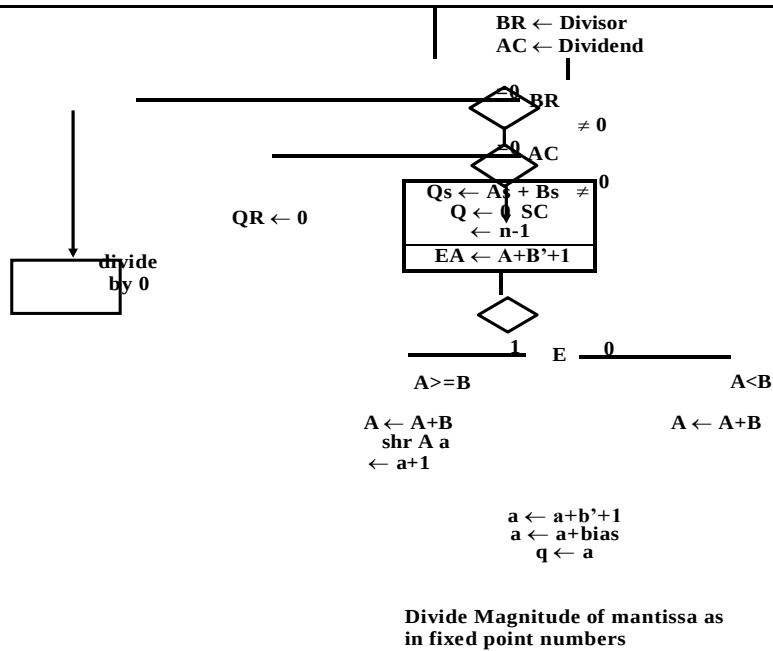
Prof. H. Yoon

Computer Arithmetic

17

Floating Point Arithmetic

FLOATING POINT DIVISION



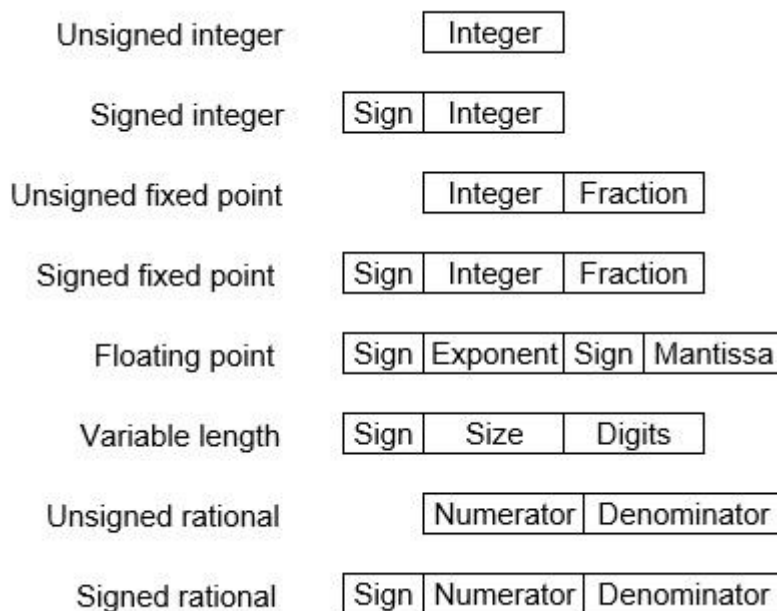
Fixed Point and Floating Point Number Representations

Digital Computers use **Binary number system** to represent all types of information inside the computers. Alphanumeric characters are represented using binary bits (i.e., 0 and 1). Digital representations are easier to design, storage is easy, accuracy and precision are greater.

There are various types of number representation techniques for digital number representation, for example: Binary number system, **octal number system**, **decimal number system**, and **hexadecimal number system** etc. But Binary number system is most relevant and popular for representing numbers in digital computer system.

Storing Real Number

These are structures as following below –



There are two major approaches to store real numbers (i.e., numbers with fractional component) in modern computing. These are (i) Fixed Point Notation and (ii) Floating Point Notation. In fixed point notation, there are a fixed number of digits after the decimal point, whereas floating point number allows for a varying number of digits after the decimal point.

Fixed-Point Representation –

This representation has fixed number of bits for integer part and for fractional part. For example, if given fixed-point representation is IIII.FFFF, then you can store minimum value is 0000.0001 and maximum value is 9999.9999. There are three parts of a fixed-point number representation: the sign field, integer field, and fractional field.

1	00000101	101011000000000000000000
Sign bit	Exponent part	Mantissa part

Where 00000101 is the 8-bit binary value of exponent value +5.

Note that 8-bit exponent field is used to store integer exponents $-126 \leq n \leq 127$.

The smallest normalized positive number that fits into 32 bits is $(1.000000000000000000000000)_2 \times 2^{-126} = 2^{-126} \approx 1.18 \times 10^{-38}$, and largest normalized positive number that fits into 32 bits is $(1.111111111111111111111111)_2 \times 2^{127} = (2^{24}-1) \times 2^{104} \approx 3.40 \times 10^{38}$. These numbers are represented as following below,

Smallest	0	10000010	000000000000000000000000
	Sign bit	Exponent part	Mantissa part
Largest	0	01111111	111111111111111111111111
	Sign bit	Exponent part	Mantissa part

The precision of a floating-point format is the number of positions reserved for binary digits plus one (for the hidden bit). In the examples considered here the precision is $23+1=24$.

The gap between 1 and the next normalized floating-point number is known as machine epsilon. the gap is $(1+2^{-23})-1=2^{-23}$ for above example, but this is same as the smallest positive floating-point number because of non-uniform spacing unlike in the fixed-point scenario.

Note that non-terminating binary numbers can be represented in floating point representation, e.g., $1/3 = (0.010101 \dots)_2$ cannot be a floating-point number as its binary representation is non-terminating.

IEEE Floating point Number Representation –

IEEE (Institute of Electrical and Electronics Engineers) has standardized Floating-Point Representation as following diagram.

